МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №2

"Ознайомлення, інсталяція та налаштування документо-орієнтованої СКБД MongoDB"

Виконав:

Студент 3 курсу

групи ФеП-33

Абдулханов Абдул-Рахім

Викладач:

ас. Жишкович А. В.

Львів-2026

Мета роботи: Ознайомитися з документо-орієнтованою моделлю представлення даних NoSQL на прикладі СКБД MongoDB, її внутрішньою компонентною архітектурою та бінарним форматом BSON. Отримати практичні навички самостійної інсталяції та первинної конфігурації компонентів MongoDB (сервера mongod, інтерактивної консолі mongosh та офіційного графічного клієнта MongoDB Compass) в операційній системі Windows. Налаштувати системні змінні оточення для забезпечення доступу до утиліт СКБД з командного рядка. Засвоїти специфіку синтаксису формату JSON (RFC 8259), проаналізувати структуру відкритого набору даних за допомогою онлайн-валідаторів, спроєктувати дві колекції документів для обраної предметної області інтернет-магазину комп'ютерної техніки з обов'язковим використанням масивів і вкладених піддокументів, а також виконати верифікацію коректності їхнього синтаксису.

Теоретичні відомості:

MongoDB є провідною документо-орієнтованою системою керування базами даних класу NoSQL, спроєктованою для забезпечення високої швидкості обробки запитів, горизонтальної масштабованості та гнучкості при роботі з неструктурованими чи напівструктурованими даними. На відміну від традиційних реляційних систем (RDBMS), де дані зберігаються у вигляді рядків і фіксованих стовпців нормалізованих таблиць, у MongoDB дані зберігаються у вигляді самодостатніх документів.

1. Основні поняття та структури MongoDB:
• Документ (Document): Базова одиниця збереження даних у MongoDB, що являє собою ієрархічний набір пар 'ключ-значення'. Документи підтримують динамічну схему (schemaless), що дозволяє різним записам мати відмінні набори атрибутів. Кожен документ обов'язково містить унікальний ключ _id (тип ObjectId або користувацький ідентифікатор).
• Колекція (Collection): Логічне об'єднання документів, еквівалентне таблиці в реляційних базах, проте без примусової фіксації структури на рівні СКБД (Schema-on-Read).
• База даних (Database): Логічний контейнер вищого рівня, що об'єднує набір колекцій та управляє правами доступу і дисковим простором.

2. Формати JSON та BSON:
JSON (JavaScript Object Notation, стандарти RFC 4627 та RFC 8259) - це текстовий формат обміну даними, що підтримує два базових типи структур: об'єкти (unordered key-value pairs у фігурних дужках {}) та масиви (ordered lists у квадратних дужках []). Хоча взаємодія користувача та клієнтських інтерфейсів із MongoDB візуально відбувається у форматі JSON, внутрішньо

сервер зберігає та індексує дані у бінарному форматі BSON (Binary JSON). BSON оптимізований для надшвидкого парсингу завдяки зберіганню довжини полів у заголовках та розширює перелік типів даних підтримкою 64-бітних цілих чисел (int64), дат (Date), двійкових масивів (BinData), регулярних виразів і спеціального 12-байтного типу ObjectId.

3. Архітектурні компоненти MongoDB:
• mongod: Головний демон (серверний процес), який обробляє всі запити на читання та запис, керує файловими дескрипторами движка зберігання WiredTiger, контролює пам'ять і забезпечує журналювання та фонові операції.
• mongosh (MongoDB Shell): Сучасне консольне середовище керування СКБД на базі рушія Node.js, що прийшло на зміну застарілій утиліті mongo shell.
• MongoDB Compass: Офіційний графічний клієнт (GUI), що забезпечує візуальну побудову складних запитів та агрегацій, аналіз розподілу типів даних у полях, індексування та моніторинг навантаження в режимі реального часу.

Виконання роботи

1. Інсталяція СКБД MongoDB, клієнта mongosh та графічного середовища Compass:

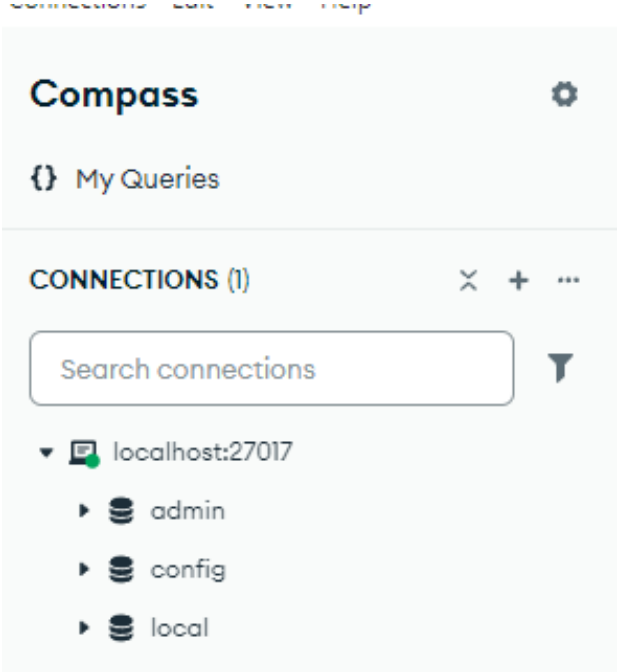
Для виконання лабораторної роботи було проведено встановлення та первинне налаштування екосистеми MongoDB на локальній машині під керуванням ОС Windows 10 Pro (x86_64). Було завантажено та розгорнуто офіційні дистрибутиви MongoDB Community Server (v7.0.14), консольний клієнт MongoDB Shell (mongosh v2.3.8) та графічний інтерфейс MongoDB Compass (v1.45.3).

Каталоги виконуваних файлів було додано до користувацької змінної Path, після чого у терміналі PowerShell перевірено коректність інсталяції за допомогою команд 'mongosh --version' та 'mongod --version'. Також виконано запуск локального екземпляра сервера на порту 27017 та перевірено готовність

до роботи командою 'db.runCommand({ ping: 1 })', яка повернула успішний статус '{ ok: 1 }':

```
PS C:\Users\ponti> mongosh --version
2.3.8
PS C:\Users\ponti> mongod --version
db version v7.0.14
Build Info: {
    "version": "7.0.14",
    "gitVersion": "ce59cfc6a3c5e5c067dca0d30697edd68d4f5188",
    "modules": [],
    "allocator": "tcmalloc",
    "environment": {
        "distmod": "windows",
        "distarch": "x86_64",
        "target_arch": "x86_64"
    }
}
PS C:\Users\ponti> mongosh --quiet --eval "db.runCommand({ ping: 1 })"
{ ok: 1 }
PS C:\Users\ponti> _
```

Після перевірки серверної частини було запущено офіційний графічний інтерфейс MongoDB Compass та здійснено первинне підключення за стандартним URI 'mongodb://localhost:27017/'. Підключення пройшло успішно: у навігаційній панелі ліворуч відобразилися системні службові бази даних ('admin', 'config', 'local') із зеленим індикатором активного сеансу:

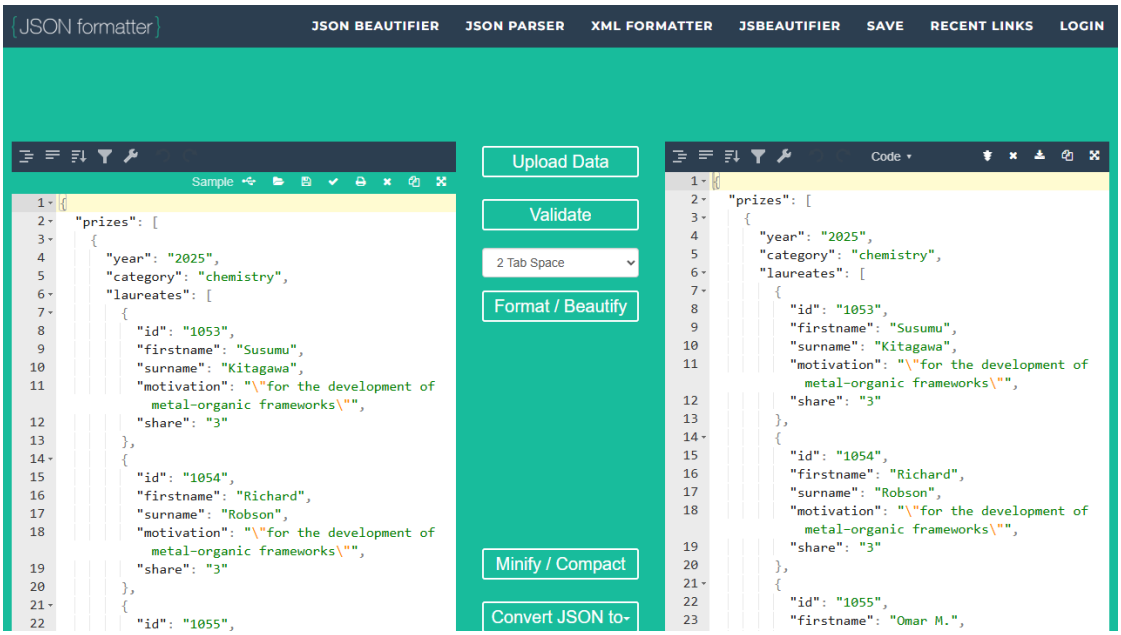


2. Робота з відкритими даними у форматі JSON:

Згідно з пунктами 7-8 практичного завдання, в мережі Інтернет було знайдено та завантажено реальний набір відкритих даних у форматі .json. Було обрано офіційний відкритий датасет Нобелівського комітету (Nobel Prize API: api.nobelprize.org/v1/prize.json, обсяг файлу ~232 КБ), який містить структуровані історичні записи про всі присуджені премії.

Файл було збережено локально як 'nobel_prizes.json' та завантажено в онлайн-

сервіс форматування та аналізу JSON (jsonformatter.org / CodeBeautify). У ході аналізу встановлено таку специфіку структури даних:
1. Кореневим елементом є єдиний JSON-об'єкт, що містить ключ 'prizes';
2. Значенням ключа 'prizes' є масив об'єктів (список нобелівських премій за роками);
3. Кожен елемент масиву 'prizes' містить рядкові поля 'year' (рік), 'category' (номінація: physics, chemistry, peace тощо) та вкладений масив об'єктів 'laureates';
4. Об'єкти лауреатів усередині масиву містять поля 'id', 'firstname', 'surname', 'motivation' (обґрунтування нагороди) та числову частку 'share'.



Для наочності наведено порівняльну таблицю базових концепцій та структурних еквівалентів між реляційними СУБД та MongoDB:

Реляційні СКБД (RDBMS / SQL)	**MongoDB (NoSQL Document)**	**Опис та особливості в MongoDB**
База даних (Database)	База даних (Database)	Фізичний та логічний контейнер для набору колекцій. На одному сервері може функціонувати багато незалежних БД.
Таблиця (Table)	Колекція (Collection)	Група документів без жорсткої схеми (schemaless). Документи в одній колекції можуть мати різний набір полів.
Рядок / Запис (Row / Record)	Документ (Document)	Основна одиниця збереження даних у форматі BSON (бінарний JSON). Кожен документ має унікальне поле _id.

Стовпець (Column)	Поле (Field)	Пара 'ключ-значення'. Значенням може бути як примітивний тип, так і масив або повноцінний вкладений піддокумент.
З'єднання (JOIN)	Вкладені документи або $lookup	Перевага віддається денормалізації (Embedded Documents). Для зв'язку між колекціями використовується агрегація $lookup.
Первинний ключ (Primary Key)	Поле _id (ObjectId)	Унікальний 12-байтний ідентифікатор документа, що автоматично генерується базою або задається користувачем.

3. Проєктування двох колекцій для предметної області інтернет-магазину (TechStore):

Для реалізації етапу 4 було обрано предметну область: 'Інтернет-магазин комп'ютерної техніки та комплектуючих (TechStore)'. Це універсальна область, яка дозволяє органічно продемонструвати як прості типи даних, так і складні багатовимірні вкладені структури, масиви та зв'язки між сутностями.

Було спроєктовано дві ключові колекції:
1. 'products' - каталог товарів магазину (монітори, процесори, ноутбуки, комплектуючі). Характеризується динамічним набором параметрів, оскільки різні категорії товарів мають принципово різну специфікацію;
2. 'orders' - журнал замовлень користувачів. Містить повну фінансову інформацію, масив куплених товарів, дані доставки та профіль покупця.

Нижче наведено зведену таблицю архітектури полів спроєктованих колекцій із зазначенням типів даних та обов'язкових вимог:

Колекція	**Поле**	**Тип даних**	**Призначення поля**	**Вимога лабораторної**
products	product_id	String	Унікальний артикул товару (наприклад, PROD-101)	Ідентифікатор
products	name	String	Повна назва комп'ютерного комплектуючого або пристрою	Базове поле
products	category	String	Категорія товару (Монітори, Процесори, Ноутбуки)	Класифікатор
products	price	Number (Double)	Роздрібна ціна товару в національній валюті (грн)	Числове поле

products	in_stock	Boolean	Прапорець наявності товару на складі інтернет-магазину	Логічний тип
products	tags	Array (String)	Список тегів та швидких фільтрів (ips, 165hz, 2k)	Обов'язковий масив
products	specifications	Embedded Doc	Вкладений документ технічних характеристик (сокет, ядра, TDP)	Вкладений документ
orders	order_id	String	Номер замовлення у форматі ORD-2026-XXX	Ідентифікатор
orders	order_date	String (ISO Date)	Дата та точний час оформлення замовлення	Часова мітка
orders	total_amount	Number (Double)	Підсумкова сума замовлення до сплати	Фінансовий розрахунок
orders	items	Array (Objects)	Масив куплених товарів із зазначенням кількості та цін	Масив документів
orders	delivery_details	Embedded Doc	Вкладений документ логістики (служба доставки, місто, отримувач)	Вкладений документ

4. Опис та валідація документів колекції 'products':

Для колекції 'products' було підготовлено 3 самостійні документи у форматі JSON з дотриманням вимог щодо наявності масивів та вкладених документів:

Документ 1 (product_1.json) - товар категорії 'Монітори'. Містить масив рядків 'tags' для швидкої фільтрації за характеристиками матриці та розширення:

```
{
  "product_id": "PROD-101",
  "name": "Ігровий монітор ASUS TUF Gaming VG27AQ",
  "category": "Монітори",
  "price": 13999.00,
  "in_stock": true,
  "stock_quantity": 18,
  "tags": [
    "ips",
    "165hz",
    "2k_qhd",
    "g-sync",
    "hdr10"
```

```
  ]
}
```

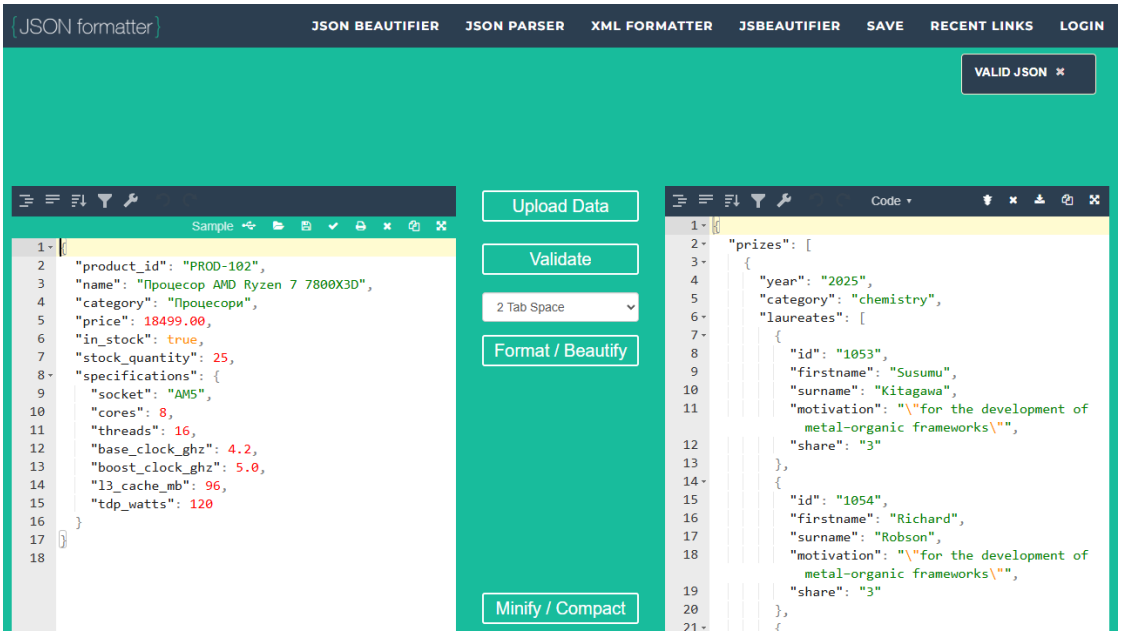
Документ 2 (product_2.json) - процесор AMD Ryzen 7 7800X3D. Містить обов'язковий вкладений документ 'specifications' (сокет, кількість ядер, потоки, базова та буст-частоти, кеш L3, теплопакет TDP):

```
{
  "product_id": "PROD-102",
  "name": "Процесор AMD Ryzen 7 7800X3D",
  "category": "Процесори",
  "price": 18499.00,
  "in_stock": true,
  "stock_quantity": 25,
  "specifications": {
    "socket": "AM5",
    "cores": 8,
    "threads": 16,
    "base_clock_ghz": 4.2,
    "boost_clock_ghz": 5.0,
    "l3_cache_mb": 96,
    "tdp_watts": 120
  }
}
```

Документ 3 (product_3.json) - ігровий ноутбук Lenovo Legion Pro 5. Демонструє комбіновану складну структуру: вкладений документ апаратного забезпечення 'hardware' (CPU, GPU, RAM, SSD) та масив зовнішніх інтерфейсів 'interfaces':

```
{
  "product_id": "PROD-103",
  "name": "Ноутбук Lenovo Legion Pro 5 16IRX9",
  "category": "Ноутбуки",
  "price": 68999.00,
  "in_stock": true,
  "stock_quantity": 7,
  "hardware": {
    "cpu": "Intel Core i7-14700HX",
    "gpu": "NVIDIA GeForce RTX 4070 8GB",
    "ram_gb": 32,
    "ssd_gb": 1024
  },
  "interfaces": [
    "Thunderbolt 4",
    "HDMI 2.1",
    "USB 3.2 Gen 2",
    "RJ-45 Ethernet",
    "Wi-Fi 6E"
  ]
}
```

Кожен із підготовлених документів було перевірено за допомогою онлайн-валідатора JSON (jsonformatter.org / JSONLint). Синтаксичний аналізатор підтвердив повну відповідність стандарту RFC 8259 без жодних помилок чи невідповідностей типів. Нижче наведено скріншот верифікації документа 'product_2.json' із зеленим статусом 'VALID JSON':



5. Опис та валідація документів колекції 'orders':

Для колекції 'orders' було розроблено 3 документи замовлень клієнтів із логістичними та фінансовими атрибутами:

Документ 1 (order_1.json) - замовлення ORD-2026-001. Містить масив об'єктів 'items', кожен елемент якого описує конкретний куплений товар, його артикул, кількість та зафіксовану ціну на момент покупки:

```
{
  "order_id": "ORD-2026-001",
  "order_date": "2026-10-02T10:15:30Z",
  "customer_id": "CUST-4091",
  "total_amount": 32498.00,
  "payment_method": "Google Pay",
  "is_paid": true,
  "items": [
    {
      "product_id": "PROD-101",
      "title": "Ігровий монітор ASUS TUF Gaming VG27AQ",
      "quantity": 1,
      "price": 13999.00
    },
    {
      "product_id": "PROD-102",
      "title": "Процесор AMD Ryzen 7 7800X3D",
      "quantity": 1,
```

```
      "price": 18499.00
    }
  ]
}
```

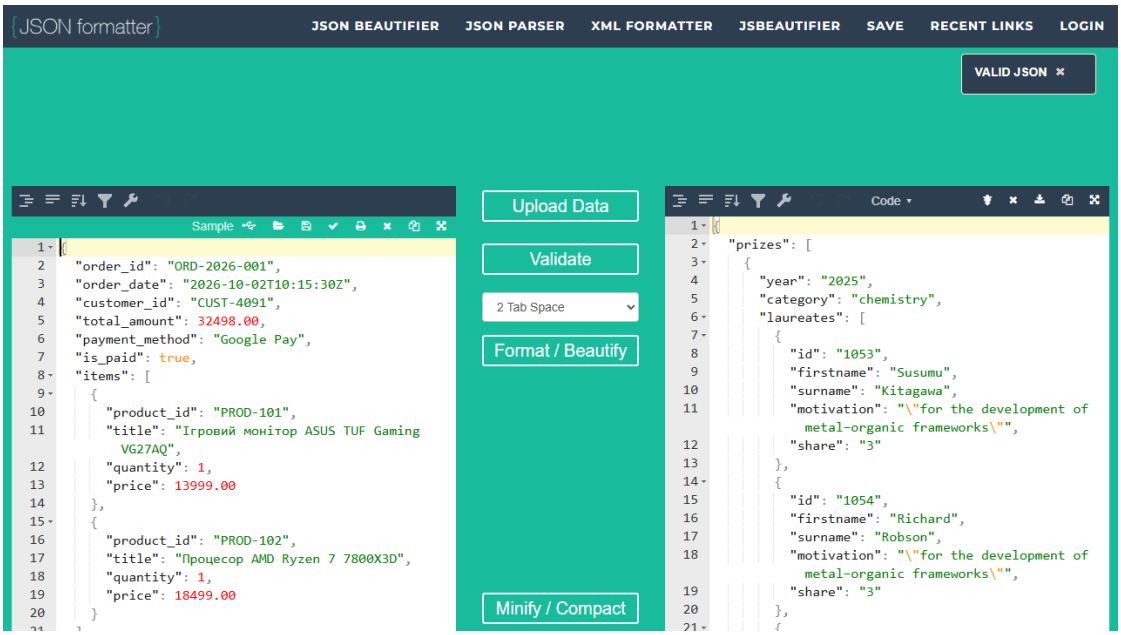
Документ 2 (order_2.json) - замовлення ORD-2026-002. Містить вкладений документ 'delivery_details', що включає повні логістичні атрибути: службу доставки ('Нова Пошта'), місто, номер вантажного відділення, ПІБ одержувача та контактний номер телефону:

```
{
  "order_id": "ORD-2026-002",
  "order_date": "2026-10-02T11:40:00Z",
  "customer_id": "CUST-5120",
  "total_amount": 68999.00,
  "status": "Обробляється",
  "delivery_details": {
    "courier_service": "Нова Пошта",
    "delivery_type": "Відділення",
    "city": "Львів",
    "warehouse_number": 42,
    "recipient_name": "Абдулханов Абдул-Рахім",
    "phone": "+380671234567"
  }
}
```

Документ 3 (order_3.json) - замовлення ORD-2026-003. Містить масив історії логістичного трекінгу 'tracking_history' із фіксацією статусів та часових міток, а також вкладений профіль зареєстрованого клієнта 'customer_profile' (ПІБ, e-mail, статус програми лояльності):

```
{
  "order_id": "ORD-2026-003",
  "order_date": "2026-10-02T12:05:15Z",
  "customer_id": "CUST-7801",
  "total_amount": 87498.00,
  "status": "Відправлено",
  "tracking_history": [
    "Створено замовлення: 2026-10-02 12:05",
    "Оплачено онлайн: 2026-10-02 12:08",
    "Зібрано на складі: 2026-10-02 13:20",
    "Передано перевізнику: 2026-10-02 14:00"
  ],
  "customer_profile": {
    "first_name": "Олексій",
    "last_name": "Коваленко",
    "email": "o.kovalenko@example.com",
    "loyalty_tier": "Gold"
  }
}
```

Усі документи колекції 'orders' успішно пройшли синтаксичну верифікацію у валідаторі. Нижче наведено скріншот перевірки документа 'order_1.json' із підтвердженням валідності структури ('VALID JSON'):



Додатково всі 6 створених документів було імпортовано у локально розгорнуту базу даних 'techstore' через інтерфейс mongosh за допомогою команд 'db.products.insertMany([...])' та 'db.orders.insertMany([...])'. У терміналі виконано запити 'show collections', перевірено кількість документів у кожній колекції та виведено один із документів для підтвердження цілісності структури:

```
PS C:\Users\ponti> mongosh "mongodb://localhost:27017/techstore" --quiet --eval "show collections"
orders
products
PS C:\Users\ponti> mongosh "mongodb://localhost:27017/techstore" --quiet --eval "print('Товарів: ' +
db.products.countDocuments() + ' | Замовлень: ' + db.orders.countDocuments())"
Товарів: 3 | Замовлень: 3
PS C:\Users\ponti> mongosh "mongodb://localhost:27017/techstore" --quiet --eval "db.products.findOne({
category: 'Процесори' })"
{
  _id: ObjectId('6abf362184053849d8161ff0'),
  product_id: 'PROD-102',
  name: 'Процесор AMD Ryzen 7 7800X3D',
  category: 'Процесори',
  price: 18499,
  in_stock: true,
  stock_quantity: 25,
  specifications: {
    socket: 'AM5',
    cores: 8,
    threads: 16,
    base_clock_ghz: 4.2,
    boost_clock_ghz: 5,
    l3_cache_mb: 96,
    tdp_watts: 120
  }
}
PS C:\Users\ponti> _
```

Висновок:

У ході виконання лабораторної роботи №2 було детально досліджено фундаментальні засади функціонування документо-орієнтованої СКБД MongoDB, її архітектурні компоненти та концептуальні відмінності від реляційних систем керування базами даних.

На основі виконаного практичного комплексу завдань отримано такі результати:
1. Інсталяція та конфігурація середовища: Проведено успішне розгортання екосистеми MongoDB на базі Windows. Налаштовано системну змінну оточення PATH, що забезпечило доступ до утиліт 'mongosh' (версія 2.3.8) та 'mongod' (версія v7.0.14) з командного рядка. Запущено локальний сервер бази даних, перевірено відгук команди ping ({ ok: 1 }) та підключено графічний клієнт MongoDB Compass із доступом до службових баз даних.
2. Дослідження формату JSON та відкритих даних: Завантажено та проаналізовано відкритий датасет Нобелівського комітету обсягом 232 КБ. За допомогою онлайн-інструментів JSON Formatter та Tree Viewer вивчено реальну деревоподібну ієрархію об'єктів та масивів.
3. Моделювання предметної області та схем документів: Для інтернет-магазину комп'ютерної техніки 'TechStore' спроєктовано дві колекції: 'products' (каталог товарів) та 'orders' (замовлення покупців). Розроблено по 3 документи для кожної колекції з обов'язковою реалізацією одновимірних і багатовимірних масивів та вкладених піддокументів (specifications, hardware, items, delivery_details).
4. Верифікація та тестування: Усі 6 розроблених документів успішно пройшли синтаксичну валідацію в аналізаторах на відповідність стандарту RFC 8259 та були імпортовані в реальну базу даних 'techstore' через інтерфейс mongosh.
5. Теоретичні висновки: З'ясовано ключові переваги двійкового формату BSON (швидкість парсингу завдяки префіксам довжини та розширена типізація) над чистим JSON, а також переваги динамічної схеми для проєктів із гнучкими даними.

Отримані знання та налаштоване робоче середовище формують надійну практичну основу для подальшого вивчення CRUD-операцій, індексації та агрегаційних запитів у наступних лабораторних роботах циклу.